

## G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

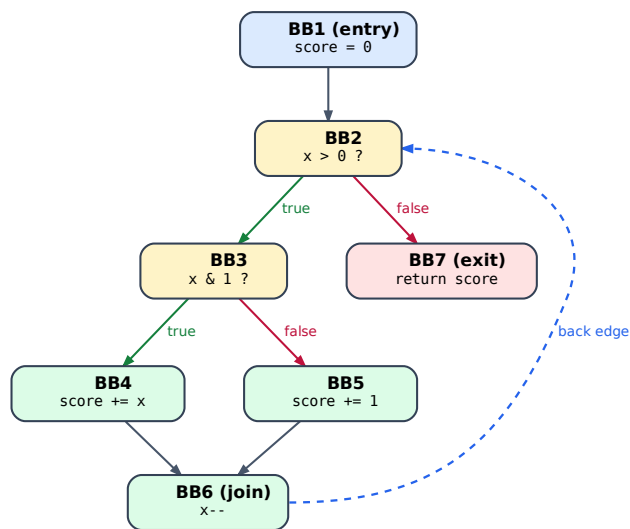


Figure 1: 一個含迴圈與條件分支的 CFG。每個框是一個 basic block； $BB_6$  是兩條分支的 join point， $BB_6 \rightarrow BB_2$  是迴圈的 back edge。

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 在程式執行期間總共被通過幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有  $N$  個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從  $BB_i$  可以走到  $BB_j$ ，總共有  $M$  條有向邊。

程式每執行一次，控制流都會從入口 1 開始，沿著 CFG 上的有向邊前進，最後抵達出口  $N$  結束執行。過程中每沿著一條有向邊走過一次，該邊的通過次數就增加 1；若因為迴圈而在同一次執行中重複通過同一條邊，則每次通過都會分別計數。程式總共執行  $R$  次，令  $f_e$  表示這  $R$  次執行合計通過邊  $e$  的次數。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 ( $BB_i$  或  $BB_j$  都可以，編譯器只會選一個 BB 插入)，因為測量通過次數會造成額外的運算，所以插入每個計數器需要成本  $c_e$ ，執行結束後就能知道該邊通過的次數。

每次控制流進入一個普通 BB 後，都一定會再沿著某條邊離開。因此，普通 BB 被進入的總次數會等於它被離開的總次數。入口 1 是每次執行的起點，所以離開次數會比進入次數多  $R$ ；出口  $N$  則正好相反。也就是說：

- 對於入口與出口以外的每個 BB，所有連進該 BB 的邊，其通過次數總和，等於所有從該 BB 連出去的邊，其通過次數總和。

- 對於入口  $1$ ，所有連出去的邊的通過次數總和，等於所有連進來的邊的通過次數總和加上  $R$ 。
- 對於出口  $N$ ，所有連進來的邊的通過次數總和，等於所有連出去的邊的通過次數總和加上  $R$ 。

利用這些進出次數的關係，可以從已知的邊通過次數推導出部分沒有安裝計數器的邊。

編譯器必須在程式執行前決定安裝計數器的邊。無論這  $R$  次執行分別採取哪些合法路徑，執行結束後都必須能由  $R$ 、CFG、進出次數守恆關係與所有計數器的讀值，唯一確定每條邊的通過次數。求最小安裝成本，使得執行結束後能夠**唯一確定每一條邊通過的次數**。

### — 輸入 —

第一行會有三個整數  $N, M, R$ ，代表 BB 的數量、邊的數量與執行次數

接下來  $M$  行會有三個整數  $u, v, C$ ，代表  $BB_u$  可以走到  $BB_v$ ，且安裝計數器成本為  $C$

### — 輸出 —

最小安裝成本

### — 輸入限制 —

- $2 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到  $N$  的路徑通過

### — 子任務 —

| 編號 | 分數 | 額外限制                                           |
|----|----|------------------------------------------------|
| 1  | 0  | 範例輸入輸出                                         |
| 2  | 3  | $M = N - 1$ , 且第 $i$ 條邊為 $i \rightarrow i + 1$ |
| 3  | 7  | $C_e = 1$                                      |
| 4  | 13 | $M \leq 20$                                    |
| 5  | 19 | $M = N$ 且無 self-loop 或重複連接                     |
| 6  | 21 | 每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2               |
| 7  | 37 | 無額外限制                                          |

### — 範例輸入 —

4 4 8  
1 2 5  
1 3 2  
2 4 4  
3 4 6

### — 範例輸出 —

2

### — 範例解釋 —

輸入的 CFG 的結構如下：

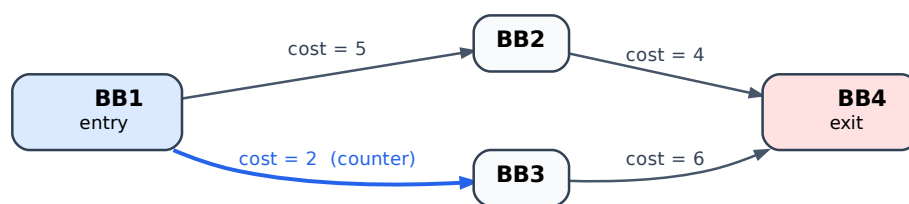


Figure 2: 範例一的 CFG。邊上標示安裝計數器的成本；藍色粗邊表示在  $1 \rightarrow 3$  安裝計數器。

程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$  或  $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊  $1 \rightarrow 3$  上安裝計數器。

假設程式執行完畢後，計數器顯示邊  $1 \rightarrow 3$  總共被通過 3 次。因為程式總共執行 8 次，所以邊  $1 \rightarrow 2$  必定被通過  $8 - 3 = 5$  次。

每次進入  $BB_2$  後都會沿著邊  $2 \rightarrow 4$  離開，因此邊  $2 \rightarrow 4$  也被通過 5 次。

同理，邊  $3 \rightarrow 4$  被通過 3 次。

因此，只要在邊  $1 \rightarrow 3$  上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過  $BB_2$  或  $BB_3$ 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

### — 範例輸入 —

4 7 14  
1 2 8  
2 4 5  
1 3 7

3 4 4

2 3 6

2 2 3

1 2 9

— 範例輸出 —

21

— 範例輸入 —

6 6 12

1 2 9

2 3 8

3 4 3

2 4 7

4 5 6

5 6 5

— 範例輸出 —

3

— 範例輸入 —

8 9 18

1 2 10

2 3 8

3 5 2

2 4 6

4 5 5

5 6 7

6 7 4

7 8 3

6 8 9

— 範例輸出 —

5

## － 編譯器小教室：從 C Code 到 CFG －

C 的 if、while、for 很適合人看，但對編譯器來說，這些語法還是太有結構了。做最佳化時，通常會先把它們拆成幾種比較單純的控制流操作：

```
goto L
if condition goto L_true else goto L_false
return value
```

把原本的結構化控制流拆成 label 和 jump，通常叫做 **lowering**。

下面使用的 goto-style 是教學用的 pseudo IR。它和 LLVM IR 一樣，會用 label 和明確的 branch 表示控制流，但它不是合法的 **LLVM IR**。真正的 LLVM IR 還有型別、SSA、phi 等規則，而且 terminator 使用的是 br、ret、switch 等指令。

兩者可以先這樣理解：

```
goto L
    <=> br label %L

if c goto T else goto F
    <=> br i1 %c, label %T, label %F

return x
    <=> ret i32 %x
```

把大括號裡的 if、迴圈等結構拆成 label 和 goto，概念上就是 TOJ 756 在做的事情。

### Basic Block (BB)

Basic block 是一段**最大且連續**的指令序列。它有幾個基本條件：

- 控制流只能從 block 的第一條指令進入，不能跳到中間
- block 執行到最後之前，不會跳到其他地方。
- 一旦進入 block，就會依序執行裡面的指令，直到遇到 terminator。

在 LLVM-like IR 中，每個 BB 從 label 開始，最後以一條 terminator 結束。terminator 會決定下一個 BB，或者直接結束函式。

如果現在手上只有一串 goto-style 指令，可以先找 **leaders**：

1. 函式的第一條指令是 leader。
2. 每個 jump 的目標 label 是 leader。
3. if、goto、return 後面如果還有指令，那條下一條指令也是 leader。

找到 leaders 之後，從一個 leader 開始，一直到下一個 leader 前的指令，都屬於同一個 BB。

實作時通常還會再處理 unreachable blocks，並視情況合併不必要的空 block。

### 從 BB 建立 CFG

CFG 的每個節點就是一個 BB。

如果執行完  $BB_u$  後，下一步可能執行  $BB_v$ ，就在圖上加一條有向邊  $u \rightarrow v$ 。

- goto BB2：加入一條邊到  $BB_2$ 。
- if c goto BB2 else BB3：加入  $BB_2$  和  $BB_3$  兩條邊。
- return：沒有函式內的 successor。
- 如果 IR 允許 fall-through，就把邊接到文字順序的下一個 BB。

本文會把 jump 都直接寫出來，所以不考慮隱藏的 fall-through。

也就是說，前面兩個步驟可以分開看：label 決定 BB 怎麼切，terminator 決定 CFG 的邊。

## 常見 C 控制結構如何 Lower ?

下面先把 C 程式改成 goto-style，再從 terminator 直接看 CFG。

### 1. 直線程式

```
x = a + b;  
y = x * 2;  
return y;
```

沒有 branch，所以整段就是一個 BB：

```
BB1:  
    x = a + b;  
    y = x * 2;  
    return y;
```

return 是 terminator。執行到這裡函式就結束了，因此後面不能再接其他可執行指令。

### 2. if / else

```
if (x < 0)  
    y = -x;  
else  
    y = x;  
use(y);
```

Lowering 後可以寫成：

```
BB1: if x < 0 goto BB2 else goto BB3  
BB2: y = -x; goto BB4  
BB3: y = x; goto BB4  
BB4: use(y); ...
```

所以 CFG 有  $1 \rightarrow 2$ 、 $1 \rightarrow 3$ 、 $2 \rightarrow 4$ 、 $3 \rightarrow 4$ 。

$BB_4$  是兩條分支重新接上的地方，也就是 join point。

如果原始程式只有 if 沒有 else，false edge 就可以直接接到後面的 join point。

### 3. while

```
while (x > 0) {  
    sum += x;  
    --x;  
}  
use(sum);
```

可以拆成：

BB1:

```
goto BB2
```

BB2:

```
if x > 0 goto BB3 else goto BB4
```

BB3:

```
sum += x;  
--x;  
goto BB2
```

BB4:

```
use(sum);  
...
```

$BB_2$  是 loop header。每次進入下一輪之前，都會在這裡重新檢查條件。

$BB_3 \rightarrow BB_2$  形成 cycle； $BB_2 \rightarrow BB_4$  則是離開迴圈的 exit edge。

#### 4. do / while

```
do {  
    read_next();  
} while (!ok);  
use_data();
```

Lowering 後：

BB1:

```
    read_next();  
    goto BB2
```

BB2:

```
    if !ok goto BB1 else goto BB3
```

BB3:

```
    use_data();  
    ...
```

這裡條件檢查在 body 後面，所以第一次一定會先執行 `read_next()`。

這就是它和 `while` 在 CFG 上最明顯的差別。

## 5. for、break 與 continue

```
for (int i = 0; i < n; ++i) {  
    if (a[i] < 0) continue;  
    if (a[i] == 0) break;  
    sum += a[i];  
}  
use(sum);
```

for (init; cond; step) body 可以拆成 init、condition、body、step 和 exit：

```
BB1: i = 0; goto BB2  
BB2: if i < n      goto BB3 else goto BB7  
BB3: if a[i] < 0   goto BB6 else goto BB4  
BB4: if a[i] == 0  goto BB7 else goto BB5  
BB5: sum += a[i]; goto BB6  
BB6: ++i; goto BB2  
BB7: use(sum); ...
```

這裡很容易寫錯 continue。

continue 應該跳到 step 的  $BB_6$ ，不能直接跳回 condition。否則  $++i$  不會執行。

break 則直接跳到 loop exit  $BB_7$ 。

**6. Short-circuit : && 與 ||**

```
if (a != 0 && b / a > 2)
    hit();
next();
```

&& 的右邊只有在左邊為 true 時才會計算，因此兩個條件會落在不同的 BB：

BB1:

```
if a != 0 goto BB2 else goto BB4
```

BB2:

```
if b / a > 2 goto BB3 else goto BB4
```

BB3:

```
hit();
goto BB4
```

BB4:

```
next():
...
```

所以當  $a == 0$  時，控制流直接從  $BB_1$  到  $BB_4$ ，不會執行  $b / a$ 。

$a || b$  則剛好反過來。左邊如果已經是 true，就不需要再算右邊；只有左邊為 false 時才會繼續。

## 7. switch

```
switch (op) {  
  case 0: y = 10; break;  
  case 1: y = 20; break;  
  default: y = -1;  
}  
use(y);
```

最簡單的 lowering 可以寫成一串比較：

```
BB1: if op == 0 goto BB3 else goto BB2  
BB2: if op == 1 goto BB4 else goto BB5  
BB3: y = 10; goto BB6  
BB4: y = 20; goto BB6  
BB5: y = -1; goto BB6  
BB6: use(y); ...
```

實際的編譯器不一定真的產生這麼多比較。也可能保留多路的 switch terminator，或者進一步做成 jump table。

但不管最後的機器碼長什麼樣，CFG 上的 dispatch block 都可能有多個 successors，各個 case 最後可以再接回共同的 join point。

## 8. 提早 return

```
BB_return_negative:  
  result = -1;  
  goto BB_exit  
  
BB_return_answer:  
  result = answer;  
  goto BB_exit  
  
BB_exit:  
  return result;
```

每個 return 都會結束目前的 BB，所以在函式內沒有 successor。

如果某個 compiler pass 希望函式只有一個出口，可以把回傳值先存起來，再全部跳到同一個出口：

這種做法常見於 CFG normalization，但不是所有 compiler 都需要這樣處理。

## 完整範例：從 C、goto-style 到 CFG

```

int calc(int x) {
    int score = 0;
    while (x > 0) {
        if (x & 1)
            score += x;
        else
            score += 1;
        --x;
    }
    return score;
}

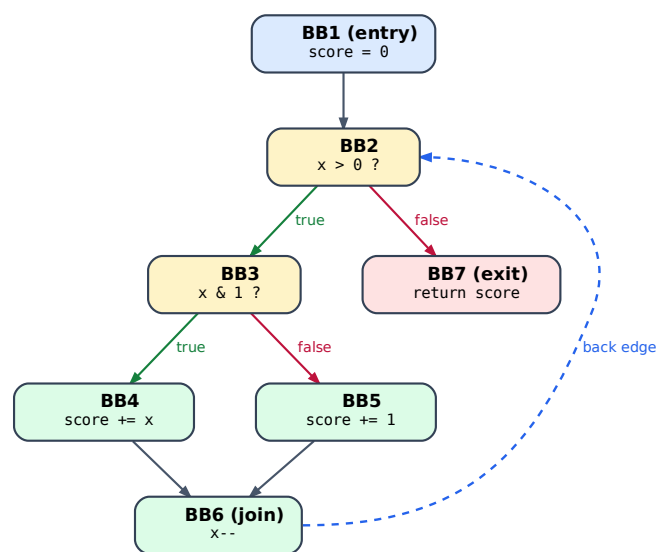
```

先把它改成只有明確 jump 的 pseudo IR：

```

BB1: score = 0; goto BB2
BB2: if x > 0 goto BB3 else goto BB7
BB3: if x & 1 goto BB4 else goto BB5
BB4: score += x; goto BB6
BB5: score += 1; goto BB6
BB6: --x; goto BB2
BB7: return score

```



上述 goto-style pseudo IR 建立出的 CFG。

$BB_4$  和  $BB_5$  是不同的 branch targets，所以不能直接合併。

接下來的  $BB_6$  則是兩條分支重新接上的地方。它有  $BB_4$ 、 $BB_5$  兩個 predecessors，因此不能把它和其中一邊直接合在一起。

最後， $BB_6 \rightarrow BB_2$  又回到 loop header，形成迴圈。

### 支配關係 (Dominance)

在有 entry 的 CFG 中，如果從 entry 到  $v$  的每一條路徑都會經過  $u$ ，就說  $u$  支配  $v$ ，記作  $u \text{ dom } v$ 。

幾個基本性質：

- 每個節點都支配自己。
- 若  $u \neq v$  且  $u$  支配  $v$ ，就是 strict dominance。
- $v$  的 immediate dominator 記為  $\text{idom}(v)$ 。它是離  $v$  最近的 strict dominator。

以前面的七個 BB 為例：

| 節點     | Dominators                   | idom   |
|--------|------------------------------|--------|
| $BB_1$ | $\{BB_1\}$                   | 無      |
| $BB_2$ | $\{BB_1, BB_2\}$             | $BB_1$ |
| $BB_3$ | $\{BB_1, BB_2, BB_3\}$       | $BB_2$ |
| $BB_4$ | $\{BB_1, BB_2, BB_3, BB_4\}$ | $BB_3$ |
| $BB_5$ | $\{BB_1, BB_2, BB_3, BB_5\}$ | $BB_3$ |
| $BB_6$ | $\{BB_1, BB_2, BB_3, BB_6\}$ | $BB_3$ |
| $BB_7$ | $\{BB_1, BB_2, BB_7\}$       | $BB_2$ |

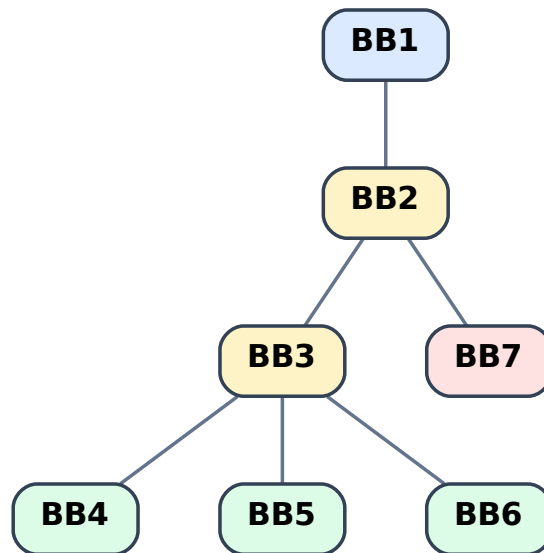
例如  $BB_4$  不支配  $BB_6$ 。因為可以走  $BB_3 \rightarrow BB_5 \rightarrow BB_6$ ，完全不經過  $BB_4$ 。

$BB_5$  也是一樣。

但不管走哪條 branch，都必須先經過  $BB_3$ ，所以  $\text{idom}(BB_6) = BB_3$ 。

## Dominator Tree

把每個非 entry 節點  $v$  接到它的  $\text{idom}(v)$ ，就會得到 dominator tree：



完整範例的 dominator tree。父節點是子節點的 immediate dominator。

在 dominator tree 裡，如果  $u$  是  $v$  的 ancestor，就代表  $u$  支配  $v$ 。

要注意的是，dominator tree 的 edge 不一定是原本 CFG 裡真的存在的 control-flow edge。它表示的是「哪些節點一定會先經過」這件事。

Dominator 會出現在很多 compiler analysis 裡。例如：

- 判斷 SSA value 的 definition 是否支配它的 uses；
- 配合 dominance frontier 決定 phi 要放在哪些 join points；
- 判斷某些運算能不能安全移動；
- 找出 loop header 和 natural loop。

## Dominance 與 Loop

對一條 CFG edge  $u \rightarrow h$ ，如果  $h$  支配  $u$ ，這條 edge 就叫做 back edge。在一般的 reducible CFG 裡， $h$  通常就是 loop header。

在上面的完整範例中， $BB_2$  支配  $BB_6$ ，所以  $BB_6 \rightarrow BB_2$  是 back edge。

這條 back edge 對應的 natural loop 是：

{  $BB_2, BB_3, BB_4, BB_5, BB_6$  }

$BB_2$  是 header,  $BB_6$  是跳回 header 的 latch;  $BB_2 \rightarrow BB_7$  則是離開迴圈的 exit edge。

因此  $BB_7$  不屬於這個 loop。

一般有向圖裡的 cycle 不一定都有一個支配整個 cycle 的 header。這種 CFG 可以是 irreducible。

相較之下, while、for 這類結構化語法通常會產生比較容易分析的 reducible CFG。

回到本題

這題已經直接給出 CFG, 所以不需要自己把 C code 再 lowering 一次。

輸入中的每個點就是一個 BB, 每條有向邊代表一種可能的控制轉移。程式執行時, 就沿著 CFG 從 entry 1 往下走, 直到 exit  $N$ 。

題目中的性能計數器統計的, 就是這些 control-flow edges 實際被走過多少次。